

Lesson 5 — Many Things At Once

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

`std::vector` — a list that grows

```
#include <vector>

std::vector<Particle> particles; // empty
particles.push_back(p);          // add to
the end
particles.size();                // how
many (UNSIGNED)
particles[3];                    // index
into it
particles.erase(particles.begin() + i);
```

An **array** has a fixed size decided at compile time. A **vector** does not.

Removing: loop BACKWARDS

```
for (int i = (int)v.size() - 1; i >= 0; i--)
{
    if (dead) v.erase(v.begin() + i);
}
```

Two traps:

- **Forwards + erase skips items** (4th time you have met this)
- **No `(int)` cast** → `size()` is unsigned, `0 - 1` wraps to ~18 quintillion, infinite loop

The proper idiom

```
#include <algorithm>
v.erase(std::remove_if(v.begin(), v.end(),
    [](const Particle& p){ return p.life <=
0; })),
    v.end());
```

Recognise it. Use the backwards loop for now.

Simulate in `double`, draw in `int`

```
struct Ball { double x, y, speed_x, speed_y;
};
put((int)ball.x, (int)ball.y, 'o');
```

`(int)` **truncates** — `3.9` → `3`. It does not round.

Write the decimal point

```
PADDLE_HEIGHT / 2.0 // right
PADDLE_HEIGHT / 2   // whole-number
division - subtly wrong
```

Mixing `int` and `double` is a top source of quiet C++ bugs.

Random directions

```
double angle = random_double(0, 2 *
3.14159);
p.speed_x = cos(angle) * speed;
p.speed_y = sin(angle) * speed;
```

Angle first → an even circle. Random x and y separately → a lumpy square.

Still true

Fix the position, then flip the velocity. (Fourth appearance.)

Section A — Recall

A1. What is the difference between an array and a `std::vector`? Give an example of when you must have a vector. [3 marks]

A2. Why must you loop *backwards* when erasing from a vector? What happens if you do not? [3 marks]

A3. Why does `(int)v.size() - 1` need the cast? What goes wrong without it? [3 marks]

A4. Why does the ball's position need to be a `double` when the screen only has whole squares? [2 marks]

A5. What does `(int)3.9` give? Is that rounding? [2 marks]

Section B — Predict the output

B1. A vector holds `[10, 20, 30, 40, 50, 60]`. This runs once. What is left, and why? [4 marks]

```
for (std::size_t i = 0; i < v.size(); i++) {  
    if (v[i] >= 30) v.erase(v.begin() + i);  
}
```

B2. The vector is **empty**. What happens? [3 marks]

```
for (std::size_t i = v.size() - 1; i >= 0; i--) { ... }
```

B3. `PADDLE_HEIGHT` is 4 (an `int`) and `paddle.y` is 6.0. What is `centre` in each version?

[3 marks]

```
double a = paddle.y + PADDLE_HEIGHT / 2.0;
double b = paddle.y + PADDLE_HEIGHT / 2;
```

Now try it with `PADDLE_HEIGHT = 5`.

B4. Two balls are at `x = 0.1` and `x = 0.9`. Where is each drawn? What does the player see as a ball crosses from 0.0 to 2.0, and is that a bug?

[4 marks]

Section C — Find and fix the bug

C1. The program compiles with a warning and then appears to hang, using 100% of a processor core. What is wrong, and what was the warning trying to tell you?

[4 marks]

```
for (int i = particles.size() - 1; i >= 0; i--) { ... }
```

C2. Roughly half the dead particles stay on screen forever. Name the bug — and say where you have met it before in this course.

[3 marks]

C3. This crashes on some machines, works on others, and sometimes works until the game has been running a while. Why?

[4 marks]

```
for (Particle& p : particles) {
    p.life -= dt;
    if (p.life <= 0) particles.erase(...);
}
```

Section D — Build it

☐ **Checkpoint 1.** Make a `std::vector<int>`, push six numbers in, print them, then remove some — **looping backwards**. Prove it works by trying forwards first and seeing what survives.

- ☐ **Checkpoint 2.** Write a `Ball` struct with `double` position and speed. Move it with `* dt` and bounce it off the top and bottom. Draw it with `(int)`.
- ☐ **Checkpoint 3.** Add two paddles. Move yours with W/S; make the opponent follow the ball **more slowly than the ball can travel**, or it can never be beaten.
- ☐ **Checkpoint 4.** Make the ball bounce off the paddles, with the hit position steering it. Remember `/ 2.0`, not `/ 2`.
- ☐ **Checkpoint 5.** Add a `std::vector<Particle>`. Spray particles on every hit, move them, fade them, and erase the dead ones.
- ☐ **Checkpoint 6.** Add scoring, a win condition, and a particle burst when a point is scored.

DO CHECKPOINT 1 THE WRONG WAY FIRST

Loop forwards while erasing, print the result, and see which items survived. Thirty seconds, and you will never write that bug again without noticing.

Section E — Design challenge

NO RIGHT ANSWER

E1. A vector keeps its items in one continuous block of memory. When it runs out of room it allocates a bigger block and **copies everything across**. What does that mean for a game spawning particles every frame? Look up `reserve()` — what does it do, and when would you call it?

E2. You have one vector of particles, and separate variables for the ball and the paddles. Why not put **everything** in one vector of "entities"? What would you gain? What would get harder?

E3. Every particle behaves identically. **Design a particle system** that supports sparks that fall, smoke that rises, and a shockwave that expands — without writing three separate systems.

E4. At 50,000 particles the game slows down. **Before optimising anything**, how would you find out where the time is actually going? Write your guess first, then say how you would check whether your guess is right.

Stretch goals

1. The ball speeds up on every paddle hit.
2. Power-ups that drift across and change the paddle size.
3. A second ball after a long rally. Would a vector of balls have been easier?
4. `reserve()` **the particles and measure whether it helps.** Be honest if it does not.
5. Colour the particles by remaining life.
6. Convert to the erase-remove idiom and check nothing changed.